

Instruction Pipelining and RISC vs. CISC

Lecture Notes — Week 12

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

September 5, 2026

12.1 The Course Arc and the Performance Question

Week 11 ended with a precise warning about overlap: parallel work is useful only when the machine preserves the meaning of the instruction stream. A five-stage pipeline can expose structural, data, and control hazards. The usual remedies—forwarding, stalls, scheduling, prediction, and recovery—trade hardware or control complexity for useful throughput. Week 12 widens the question. Pipelining is the mechanism that moves instruction work through overlapped stages; RISC and CISC are different design philosophies for describing and implementing that work.

The first discipline is to measure a program rather than infer its speed from one attractive label. For a workload with instruction count IC , average cycles per instruction CPI , and clock rate f , the CPU time is

$$T_{CPU} = \frac{IC \times CPI}{f}.$$

This is the same performance equation used in Week 1 and in Patterson and Hennessy, Ch. 1, §1.6 (p. 28) [1]. A pipeline can reduce the effective CPI of a long stream, but pipeline fill, drain, stalls, flushes, cache misses, and the instruction count remain part of the measured time. The instruction set, compiler, datapath, memory hierarchy, and workload therefore meet in the final result; no single label wins in isolation.

For the motivating two-processor comparison, Processor A completes one instruction per cycle at 2 GHz, so its CPI is 1 and its time per instruction is $1/(2 \times 10^9)$ seconds. Processor B has a 3 GHz clock but an average CPI of 2, so its time per instruction is $2/(3 \times 10^9)$ seconds. Thus A takes 0.5 ns per instruction while B takes approximately 0.667 ns: the higher clock rate does not win because B spends more cycles per instruction.

The running comparison for this chapter is the operation $A[i] = A[i] + 1$. The operation requires an element to be loaded, one to be added, and the result to be stored. An ISA determines where those operands live and how much of that work an architectural instruction names. The chapter develops four connected questions: how pipeline timing determines latency and throughput, what regularity or expressiveness buys an ISA, how a workload determines the relevant trade-off, and how the whole course can be recalled through a compact diagnostic procedure.

12.2 Pipelining: Latency, Throughput, and Hazards

12.2.1 The five-stage model

Definition (Five-stage pipeline). *The teaching model divides an instruction path into IF (instruction fetch), ID (decode and register read), EX (ALU operation or branch work), MEM*

(data memory access), and WB (register write). The same instruction visits all five stages, while different instructions occupy different stages simultaneously.

Pipeline registers separate the stages and carry an instruction's control and data state across clock boundaries. The clock period is constrained by the slowest stage plus the overhead of the pipeline registers. The five-stage functions are therefore useful for timing reasoning even though an actual processor may divide or combine them differently.

Definition (Latency, throughput, and speedup). *Latency* is the time from an instruction's IF stage to its WB stage. *Throughput* is the interval between completed results in an instruction stream. *Speedup* compares the total completion time of an overlapped implementation with the total time of a non-overlapped baseline.

The distinction matters. Once a k -stage pipeline is full, an ideal stream can complete one instruction every clock cycle, but one instruction still traverses all k stages. Thus the pipeline improves steady-state throughput; it does not make the latency of every instruction one cycle.

12.2.2 Fill, drain, and finite-stream speedup

Example 12.2.1 (Six independent instructions). Assume $n = 6$ independent instructions, $k = 5$ balanced stages, one cycle per stage, no hazards, and no extra pipeline-register cost. Without overlap, every instruction occupies all five stage-cycles, so

$$T_{seq} = nk = 6 \times 5 = 30 \text{ cycles.}$$

With overlap, the first instruction needs five cycles to pass through the pipeline. Each of the remaining five instructions enters one cycle after its predecessor, so the last instruction completes at

$$T_{pipe} = k + (n - 1) = 5 + 5 = 10 \text{ cycles.}$$

The complete schedule makes the fill and drain visible:

	1	2	3	4	5	6	7	8	9	10
I_1	IF	ID	EX	MEM	WB					
I_2		IF	ID	EX	MEM	WB				
I_3			IF	ID	EX	MEM	WB			
I_4				IF	ID	EX	MEM	WB		
I_5					IF	ID	EX	MEM	WB	
I_6						IF	ID	EX	MEM	WB

The finite-stream speedup is therefore

$$S = \frac{30}{10} = 3.0 \times .$$

The result is below the ideal fivefold speedup because this finite stream pays for filling the stages at the beginning and draining them at the end.

For a general independent stream, the same reasoning gives

$$T_{pipe} = (k + n - 1)T_c, \quad S_{ideal} = \frac{nk}{k + n - 1},$$

where T_c is the clock period. The ideal latency is approximately kT_c , whereas the ideal steady-state throughput is one completed instruction per T_c . As $n \rightarrow \infty$, $S_{ideal} \rightarrow k$ only when the stages are balanced and the stream supplies enough independent work.

12.2.3 Bubbles and the load-use dependence

Example 12.2.2 (A pipeline with one bubble). Assume that a load's data becomes available after MEM. The immediately following ALU instruction needs that value at EX, so forwarding alone cannot make the value arrive soon enough. Consider

LW R1,0(R2), ADD R3,R1,R4, SUB R5,R3,R6, OR R7,R8,R9.

The schedule is:

	1	2	3	4	5	6	7	8	9
<i>LW R1,0(R2)</i>	<i>IF</i>	<i>ID</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>				
<i>ADD R3,R1,R4</i>		<i>IF</i>	<i>ID</i>	<i>stall</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>		
<i>SUB R5,R3,R6</i>			<i>IF</i>	<i>stall</i>	<i>ID</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>	
<i>OR R7,R8,R9</i>					<i>IF</i>	<i>ID</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>

Without the bubble, the ADD would request the value before MEM had produced it. The dependence is a true read-after-write (RAW) hazard. The one bubble delays the ADD until the load result can be forwarded to its EX input; the stream therefore completes in 9 rather than the ideal 8 cycles. Its achieved CPI is shaped by the instruction mix and hazard rate, not only by the nominal number of stages.

The Socratic conclusion is consequently precise: a five-stage pipeline that completes six instructions in ten cycles has not made each instruction five times faster. The total time fell from 30 to 10 cycles, giving a 3× speedup for this finite workload. A complete performance report separates latency and throughput, then accounts for bubbles, flushes, memory stalls, and fill/drain effects.

12.2.4 Pipeline design knobs

Each design choice exchanges a possible gain for a new cost.

Choice	Potential gain	New cost	Question
Deeper pipeline	Shorter stage delay; higher f	More registers and branch penalty	Is useful work predictable?
Duplicate resources	Fewer structural hazards	Area, power, verification	Is the conflict frequent?
Forwarding	Fewer RAW stalls	Muxes, comparators, timing paths	Does the value arrive in time?
Prediction	Fewer control bubbles	Predictor state and flush recovery	What is P_{flush} ?

Stallings 11e treats instruction pipelining in §16.4 (PDF pp. 667–685); its Ch. 17 RISC discussion connects regular instruction execution to pipelining in §17.5 (PDF p. 730). These are indexed chapter and section anchors rather than claims that the constructed examples are quotations from the books.

12.3 RISC and CISC as Design Philosophies

Definition (RISC and CISC). *RISC* is a tendency toward a relatively small, regular instruction set with simple operations and a register-centered execution model. *CISC* is a tendency toward a richer instruction set in which an individual instruction may express more complex operations or addressing forms.

These are tendencies, not universal rankings. It is too strong to say that every RISC instruction is simple or every CISC instruction is slow. A fair comparison concerns the complete implementation: encoding, decode, micro-operations, compiler, memory system, and target workload. Stallings 11e discusses RISC execution, RISC pipelining, and the RISC/CISC comparison in §17.9 (PDF p. 752).

12.3.1 The running operation in two instruction streams

For $A[i] = A[i] + 1$, a register-centered or RISC tendency might make the load, add, and store explicit:

```
LOAD R1,[R2+R3]
ADD R1,R1,1
STORE [R2+R3],R1
```

A memory-rich or CISC tendency might expose a compact architectural operation such as `INC [R2+R3]`. The operation is the same, but the visible instruction stream differs. The CISC-like instruction may be split internally into simpler micro-operations before execution. This is a conceptual comparison, not a claim about one specific ISA's exact encoding. Patterson and Hennessy App. D surveys RISC addressing modes and formats (pp. D-2–D-43). The comparison can be tabulated as follows:

Register-centered / RISC tendency	Memory-rich / CISC tendency
LOAD R1,[R2+R3] ADD R1,R1,1	INC [R2+R3] <i>one architectural instruction may name the memory update</i>
STORE [R2+R3],R1	<i>decode/execute may internally split the work</i>
More instructions, regular formats, explicit loads/stores	Fewer visible instructions, richer formats, more decode work

Regularity can help a pipeline in several related ways:

- Fixed or clustered instruction formats simplify fetch boundaries and decode timing.
- A large register file reduces frequent memory operands and exposes values to forwarding paths. This is a RISC design tendency, not a law.
- Load/store separation makes the memory stage's role predictable: ALU instructions can use EX without also requesting data memory.
- Predictable stage work makes balancing and hazard detection easier, while the compiler carries more scheduling responsibility.

The cost can move into the rest of the system. More instructions can increase IC, register traffic, code size, instruction-cache pressure, or branch exposure for a particular workload.

Expressiveness can also be valuable. A complex instruction may reduce the visible instruction count for a dense operation or a legacy software interface. Rich addressing modes can express array, pointer, stack, or string operations compactly. Compatibility and code density matter when instruction memory, binary size, or an established software ecosystem dominates. The cost does not disappear: decode width, variable-length boundaries, micro-operation expansion, precise exceptions, and verification can complicate the front end.

Example 12.3.1 (Avoiding an overstrong comparison). *The following statements are reasonable tendencies:*

1. *RISC often favors regular formats and register operands.*
2. *CISC can express more work per architectural instruction.*

The statement “RISC always has fewer instructions and CISC always has lower CPI” is too strong. Instruction count, CPI, clock rate, stalls, code size, and workload interact through $T_{CPU} = IC \times CPI/f$. The appropriate comparison fixes the workload and measures total execution time rather than selecting one favorable ISA statistic.

12.4 Workload-Based Architectural Choice

Example 12.4.1 (Two workload sketches). *Consider an embedded-control workload with a tight power budget, predictable loops, limited memory, and fixed sensor operations. Its likely priorities are predictability, energy, and a compact implementation. A regular pipeline, simple decode, and compiler-visible registers may help, while code expansion and instruction-memory pressure are risks.*

Now consider cloud analytics with a large software base, diverse branches, high throughput, and frequent vector or multicore parallelism. Compatibility, scalable parallel work, and a capable front end may matter more. Decode and recovery complexity and memory bottlenecks remain risks. Patterson and Hennessy Ch. 6 emphasizes that parallel performance depends on the program and its communication and coordination costs (pp. 516–579).

The two sketches can be compared directly:

<i>Criterion</i>	<i>Embedded control</i>	<i>Cloud analytics</i>
<i>Likely priority</i>	<i>Predictability, energy, compact implementation</i>	<i>Throughput, compatibility, scalable parallel work</i>
<i>Helpful tendency</i>	<i>Regular pipeline, simple decode, compiler-visible registers</i>	<i>Strong ecosystem, capable front end, SIMD/vector and multicore support</i>
<i>Main risk</i>	<i>Code expansion and instruction-memory pressure</i>	<i>Decode/recovery complexity and memory bottlenecks</i>

Neither row proves a universal winner. Each row identifies what should be measured for the workload.

Example 12.4.2 (Compare total time, not labels). *A RISC-like implementation executes 1.20×10^9 instructions at $CPI = 1.1$ and $f = 2.4$ GHz. A CISC-like implementation executes 0.80×10^9 instructions at $CPI = 2.0$ and $f = 3.0$ GHz. Applying the performance equation gives*

$$T_R = \frac{1.20 \times 10^9 \times 1.1}{2.4 \times 10^9} = 0.55 \text{ s},$$

$$T_C = \frac{0.80 \times 10^9 \times 2.0}{3.0 \times 10^9} \approx 0.533 \text{ s}.$$

The CISC-like system wins this workload by about 3.0% despite its higher CPI, because it executes fewer instructions at a higher clock. A different workload, cache behavior, or branch rate can reverse the result. The measurement is the conclusion; the labels are hypotheses.

12.4.1 The ISA boundary and the pipeline

ISA decisions reappear as pipeline constraints.

Pipeline concern	Regular instruction stream helps	Rich instruction stream may require
Fetch/decode	Predictable boundaries and widths	Boundary detection and wider decode
Data hazards	Explicit register dependencies	Internal micro-op dependencies
Structural hazards	Regular stage/resource demand	Variable resource demand per instruction
Control recovery	Explicit branch behavior	Precise recovery across expanded work

This is the course-level connection. Week 5’s addressing modes, Weeks 6–7’s control units, Weeks 9–10’s memory hierarchy, and Week 11’s hazards all reappear as constraints on instruction throughput. The final design question is always “fit to which workload?”

12.5 Course Review and Exam Procedure

12.5.1 One performance thread through CS401

The course can be read as a sequence of increasingly complete performance questions:

1. **Representation:** bits, two’s complement, and IEEE 754 determine what values hardware can encode.
2. **Execution:** registers, buses, addressing modes, and control units move and transform those values.
3. **Communication:** I/O and interrupts connect the processor to devices without making every transfer a busy wait.
4. **Storage:** caches, virtual memory, and secondary storage trade capacity, latency, protection, and persistence.
5. **Overlap:** parallel organizations and pipelines raise throughput when hazards are identified and paid for explicitly.
6. **Choice:** the ISA and organization should be judged by the workload’s measured time, energy, reliability, and software constraints.

When a problem looks unfamiliar, ask six questions:

1. What is the representation: bits, fields, signedness, or format?
2. Where is the operand: register, memory, cache, page, or device?
3. Which hardware state changes: PC, IR, MAR/MDR, register, flag, or control word?
4. What is the bottleneck: clock, CPI, bus, memory, I/O, or hazard?

5. What is the unit of comparison: latency, throughput, speedup, hit rate, miss penalty, or total CPU time?
6. Which assumption makes the shortcut valid, and what breaks it?

A strong exam solution names the model before doing arithmetic.

Example 12.5.1 (Integrating the course topics). *Suppose a workload has many independent vector operations, frequent branches, and a large working set. It runs on a multicore processor with a pipelined front end. Inspect Week 11 and Week 12 for SIMD or MIMD opportunity, pipeline hazards, branch penalty, and instruction-stream choice. Inspect Weeks 9–10 for cache locality, TLB/page behavior, and storage misses that may dominate an otherwise fast pipeline. Finally, return to Week 1 and evaluate total time through $IC \times CPI/f$ rather than inferring performance from clock rate alone.*

For exam and assignment problems, the same procedure becomes a checklist:

- **Performance:** write the equation, normalize units, and compare total time or speedup.
- **Instruction cycle/control:** name the state and write RTL transfers with PC, IR, MAR, and MDR.
- **Memory:** split addresses into tag/index/offset or page/offset before translating or computing a hit.
- **I/O:** identify who waits, who owns the bus, and where the data moves.
- **Pipeline:** draw a cycle table; classify each hazard; count every stall and flush.
- **RISC/CISC:** state the workload, measured variables, and trade-off before defending a design choice.

The recurring course question is how a finite amount of hardware can keep common work moving quickly, correctly, and predictably. Representation makes operations possible; organization moves operands and generates control; hierarchy hides slow capacity behind fast common cases; parallelism overlaps work while hazards expose dependencies; and RISC/CISC choices shape the instruction work. The workload decides whether the trade-off pays. Therefore, do not ask which architecture is universally best. Ask which design makes this workload’s useful work per unit time highest, and state the assumptions that make the answer true.

Exercises

1. Six independent instructions run on a balanced five-stage pipeline, one cycle per stage. Compute sequential time, ideal pipelined time, finite-stream speedup, and the limiting large-stream speedup.
2. For the sequence `LW R1,0(R2)`, `ADD R3,R1,R4`, `SUB R5,R3,R6`, and `OR R7,R8,R9`, identify the true dependences, insert the required bubble, and state which result can be forwarded.
3. Explain why “RISC always has fewer instructions and CISC always has lower CPI” is not a valid universal conclusion. Name the variables that must be measured instead.
4. A RISC-like processor executes 0.90×10^9 instructions at $CPI = 1.4$ and $f = 2.1$ GHz. A CISC-like processor executes 0.60×10^9 instructions at $CPI = 2.2$ and $f = 2.8$ GHz. Compute both CPU times and identify the faster implementation for this workload.

Solutions

1. Sequential time is $6 \times 5 = 30$ cycles. Ideal pipelined time is $5 + (6 - 1) = 10$ cycles. Speedup is $30/10 = 3.0\times$. As the stream becomes very long, the ideal speedup approaches five, provided the stages are balanced and the stream supplies independent work.
2. The load produces $R1$, which ADD reads, so the first edge is a true RAW load-use dependence and requires one bubble before ADD's EX stage. ADD produces $R3$, which SUB reads; that ALU result can be forwarded to SUB without an additional bubble. The final OR is independent of those values in the shown stream.
3. The performance equation is $T_{CPU} = IC \times CPI/f$. Instruction count, CPI, clock rate, stalls, code size, cache behavior, branch behavior, and the workload interact, so neither ISA label determines total time by itself.
4. For the RISC-like processor,

$$T_R = \frac{0.90 \times 10^9 \times 1.4}{2.1 \times 10^9} = 0.60 \text{ s.}$$

For the CISC-like processor,

$$T_C = \frac{0.60 \times 10^9 \times 2.2}{2.8 \times 10^9} = 0.4714 \text{ s (approximately).}$$

The CISC-like implementation is faster for this workload because its lower instruction count and higher clock outweigh its higher CPI.

References and Source Boundaries

The deck's references are retained without changing their scope. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. (the repository key is retained as Stallings2015), is used for instruction pipelining in Ch. 16 §16.4 (PDF pp. 667–685), RISC pipelining in Ch. 17 §17.5 (PDF p. 730), the RISC/CISC comparison in §17.9 (PDF p. 752), and the brief stream-taxonomy connection in Ch. 20 §20.1 (PDF p. 847). Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM 5e, is used for the performance equation in Ch. 1 §1.6 (p. 28), the five-stage and hazard model in Ch. 4 §§4.5–4.8 (pp. 283, 297, 316, 328), workload/parallelism framing in Ch. 6 §§6.2–6.3 (pp. 518, 523), and the RISC survey in App. D (pp. D-2–D-43). The worked comparisons in this chapter are constructed examples: their numbers illustrate the performance equation and pipeline model, and are not quotations or measurements from either book.

References

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.